

Reducing Transformer Model Size with Quantization

Nafe Abubaker | 2 October 2026

Research question

Models such as BERT and LLaMA have many parameters, so storing and loading their weights can require substantial memory. Quantization reduces the number of bits used for some weights and, depending on the method, activations. This report explains the idea, compares common approaches, and reports a measured INT8 experiment on BERT. Quantization can introduce numerical error; accuracy and speed must be evaluated for the target task and hardware [1, 2].

How the numerical mapping works

For signed, symmetric, per-tensor INT8 quantization of weights w_i , one simple post-training scheme is

$$s = \frac{\max_i |w_i|}{127}, \quad q_i = \text{clip}\left(\text{round}\left(\frac{w_i}{s}\right), -127, 127\right), \quad \hat{w}_i = sq_i. \quad (1)$$

Here q_i is stored as an 8-bit integer, s is a scale, and $\hat{w}_i$ approximates the original weight. For example, with $s = 0.1$, a weight $w = 0.83$ becomes $q = 8$ and is reconstructed as $\hat{w} = 0.8$. Real systems often use separate scales per row, channel, or group to limit error [2].

For N parameters at b bits each, ideal weight storage is approximately $Nb/8$ bytes. This excludes scales, unquantized layers, activations, temporary buffers, and (for generation) the key-value cache.

Format	Bits/weight	Hypothetical 7B weight storage
FP32	32	28 GB
FP16	16	14 GB
INT8	8	7 GB
4-bit	4	3.5 GB

These decimal-GB values are arithmetic estimates for weights alone, not measured RAM requirements for a specific LLaMA release.

Methods and trade-offs

- **Post-training quantization (PTQ):** convert an already trained model without full retraining. It is straightforward to try, but the quality change must be checked. Our BERT experiment uses this approach.
- **Dynamic INT8:** quantize selected weights ahead of inference and input activations when needed. In the experiment, 73 **Linear** layers were converted; embeddings and normalization were left in higher precision [3].
- **Quantization-aware training (QAT):** simulate quantization effects during training or fine-tuning so the model can adapt; this requires additional work and training data [2].
- **4-bit LLM weights:** methods such as NF4 can reduce weight memory further. QLoRA uses a frozen 4-bit base model with trainable low-rank adapters for fine-tuning [4, 5].

Lower storage can help a model fit on available hardware. A faster response is *not* guaranteed: conversion overhead, kernels, batch size, and device support also matter [2].

Coding example 1: small FP32-to-INT8 demonstration

The accompanying `Task0_quantization_demo.py` creates a reproducible 256×256 FP32 matrix and applies Eq. 1. Its core operation is:

```
scale = np.float32(np.max(np.abs(weights)) / 127)
q = np.clip(np rint(weights / scale), -127, 127).astype(np.int8)
recovered = q.astype(np.float32) * scale
```

The simulated matrix went from 256 KiB to approximately 64 KiB including one scale (75.00% reduction). Mean absolute reconstruction error was 0.001474 and maximum error was 0.002957. This measures a weight matrix, not a complete transformer.

Coding example 2: real BERT experiment

The accompanying `Task0_quantize_bert.py` loaded `google-bert/bert-base-uncased` on a Windows CPU and used PyTorch dynamic quantization on its `Linear` modules. A short excerpt is:

```
model = AutoModel.from_pretrained("google-bert/bert-base-uncased").eval()
int8_model = torch.ao.quantization.quantize_dynamic(
    model, {torch.nn.Linear}, dtype=torch.qint8
)
```

The two `state_dict` objects were serialized to temporary files for a like-for-like checkpoint size comparison, then the files were deleted. The same input sentence was run through both models.

Observed quantity	Result
Converted <code>Linear</code> layers	73
FP32 serialized state	417.72 MiB
INT8 serialized state	173.08 MiB
Checkpoint reduction	58.56% (244.64 MiB)
Mean absolute hidden-state difference	0.229406
Maximum absolute hidden-state difference	2.435033

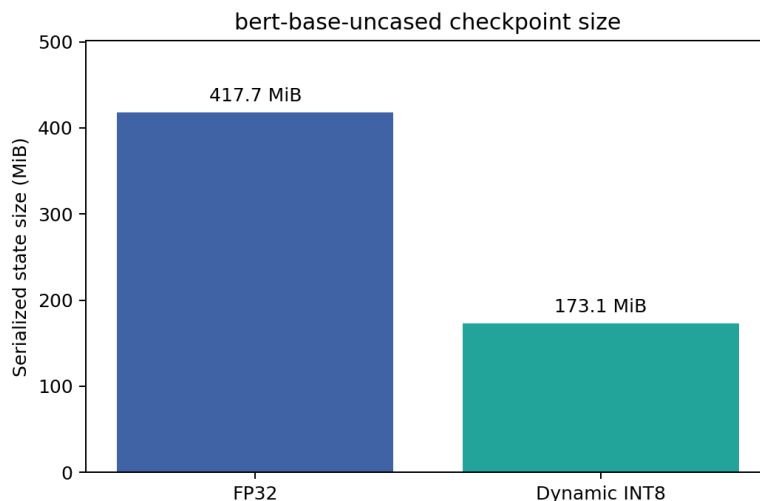


Figure 1: Measured serialized BERT states on the user’s CPU run. Whole-checkpoint reduction: 58.56%.

Interpreting the result

The output difference is a numerical comparison of one sentence’s hidden states, *not* a classification accuracy score. No labeled benchmark or latency experiment was run. A task-level study should evaluate the same validation set before and after quantization, using the relevant accuracy or F1 metric and measuring latency on the target hardware.

How the same idea applies to LLaMA

A LLaMA-family causal language model can use 4-bit weight quantization, for example with Hugging Face `BitsAndBytesConfig` and NF4 [5, 4]. This illustrative snippet was *not run* in this project:

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
import torch

config = BitsAndBytesConfig(load_in_4bit=True,
                             bnb_4bit_quant_type="nf4",
                             bnb_4bit_compute_dtype=torch.float16)
model = AutoModelForCausalLM.from_pretrained(
    "<accessible-llama-model-id>", quantization_config=config,
    device_map="auto")
```

The model identifier, access rights, installed packages, and supported hardware must be chosen for an actual run. The 3.5 GB figure above is only the ideal packed 4-bit weight storage for a hypothetical 7B model; actual memory use is higher.

Notes on the observed warnings

- The `UNEXPECTED cls.*` keys came from pretraining heads in the source checkpoint. Loading the base `BertModel` leaves those task heads unused; the encoder still loaded and produced results [6].
- `torch.ao.quantization.quantize_dynamic` completed successfully but emits a deprecation warning. PyTorch directs future eager quantization work to `torchao.quantization.quantize_` [7]. The measured script is retained for reproducibility; migration should be tested before replacing its reported values.
- The unauthenticated Hugging Face Hub warning concerns download rate limits and does not change this run’s quantization calculation.

Conclusion

INT8 post-training quantization reduced the measured BERT checkpoint from 417.72 to 173.08 MiB (58.56%). The toy matrix shows the underlying scale-and-round mechanism; the BERT run shows why whole-model savings differ from a simple bit-count estimate. Whether the quantized model retains useful task quality remains an open measurement, and 4-bit LLaMA quantization is a documented extension rather than a result measured here.

References

- [1] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR, 2018. [CVPR paper](#).
- [2] PyTorch, [torchao Quantization Overview](#) and [Quantized Inference](#).
- [3] PyTorch, [quantize_dynamic API](#).
- [4] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” 2023. [arXiv:2305.14314](#).
- [5] Hugging Face, [Bitsandbytes quantization guide](#).
- [6] Google BERT, [bert-base-uncased model card](#).
- [7] PyTorch, [migration to torchao](#).